데이터구조 3장 레포트

일체의 부정 행위 없이 스스로의 노력으로 작성한 레포트임을 확인합니다.

3.(10) 코드 및 주석

```
#include <stdio.h>
#include <memory.h>
void main()
{
char *pc; //포인터 문자열 *pc 선언
pc = (char *) malloc(1); //pc에 크기가 1을 가지는 동적 메모리를 하나 만든다.
*pc = 100;  //*pc에 100값을 넣는다.
printf("c = %d\n", *pc); //*pc의 값을 출력한다.
free(pc); //동적메모리를 해제한다.
}
```

3.(10) 출력화면 및 설명

```
C:\Users\lym69\OneDrive\바탕 화면\cording\x64\Debug\cording.exe(프로세스 15772개)이(가) 종료되었습니다(코드: -1개).
디버깅이 중지될 때 콘솔을 자동으로 닫으려면 [도구] -> [옵션] -> [디버깅] > [디버깅이 중지되면 자동으로 콘솔 닫기]를 사용하도록 설정합니다.
이 창을 닫으려면 아무 키나 누르세요...
```

동적 메모리 할당은 해서 pc가 메모리를 가지게 되고, 메모리 주소로 향하는 포인터에 100값을 넣어서 출력도 정상적으로 되는게 맞는 거지만, malloc 함수에 대한 라이브러리가 없어서 #include <stdlib.h>를 해줘야 정상적으로 코드가 실행이 된다.

```
c = 100

C:\Users\lym69\OneDrive\바탕 화면\cording\x64\Debug\cording.exe(프로세스 31676개)이(가) 종료되었습니다(코드: 0개).
디버깅이 중지될 때 콘솔을 자동으로 닫으려면 [도구] -> [옵션] -> [디버깅] > [디버깅이 중지되면 자동으로 콘솔 닫기]를 사용하도록 설정합니다.
이 창을 닫으려면 아무 키나 누르세요...
```

라이브러리를 추가해주면 c=100이라는 정상적인 출력이 된다.

3.(11) 코드 및 주석

```
#include <stdio.h>
```

```
#include <memory.h>
#include <stdlib.h>

void main()
{
	int i;
	char* pc; //포인터 문자열 *pc 선언
	pc = (char*)malloc(100);  //pc에 크기가 100을 가지는 동적 메모리를 하나 만든다.
	for (i = 0; i < 100; i++)
	{
		*pc = 100; //for문을 이용하여 *pc에 100값을 넣는 작업을 100번 반복한다.
	}
	free(pc); //동적 메모리를 해제한다.
}
```

3.(11) 출력화면 및 설명

```
C:\Users\lym69\OneDrive\바탕 화면\cording\x64\Debug\cording.exe(프로세스 29972개)이(가) 종료되었습니다(코드: 0개).
디버깅이 중지될 때 콘솔을 자동으로 닫으려면 [도구] -> [옵션] -> [디버깅] > [디버깅이 중지되면 자동으로 콘솔 닫기]를 사용
하도록 설정합니다.
이 창을 닫으려면 아무 키나 누르세요...
```

malloc을 이용하여 pc에 100 크기의 동적 메모리를 할당하였다. for 문에서 100이라는 값을 pc 메모리 하나하나에 할당하게 하려고 만든 코드이지만, 실행해보면 첫 번째 메모리에만 저장이 되는 것을 알 수 있다. 이것을 제대로 된 코드로 바꾸기 위해서는 i를 활용하여 pc[i]로 바꿔볼 수 있다.

3.(12) 코드 및 주석

```
#include <stdio.h>
#include <memory.h>
#include <stdlib.h>
void main()
{
	char* pc; //문자열 포인터 pc 선언
	int* pi; //정수형 포인터 pi 선언
	float* pf; //float형 포인터 pf 선언
	double* pd; //double형 포인터 pd 선언
	pc = (char*)malloc(100 * sizeof(char)); //크기가 100인 동적 메모리 할당
	pi = (int*)malloc(100 * sizeof(int)); //크기가 100인 동적 메모리 할당
	pf = (float*)malloc(100 * sizeof(float)); //크기가 100인 동적 메모리 할당
	pd = (double*)malloc(100 * sizeof(double)); //크기가 100인 동적 메모리 할당

	for (int i = 0; i < 100; i++) //결과값 확인을 위해서 0부터 99까지의 수를 넣는다.
	{
		pc[i] = i;
		pi[i] = i;
		pf[i] = i;
		pd[i] = i;
	}
```

```
        for (int i = 0; i < 100; i++)
        {
                printf("pc = %d  ", *(pc + i));
        }
        for (int i = 0; i < 100; i++)
        {
                printf("pi = %d  ", *(pi + i));
        }
        for (int i = 0; i < 100; i++)
        {
                printf("pf = %f  ", *(pf + i));
        }
        for (int i = 0; i < 100; i++)
        {
                printf("pd = %f  ", *(pd + i));
        }

        free(pc);
        free(pi);
        free(pf);
        free(pd);
}
```

3.(12) 출력화면 및 설명

```
pc = 0  pc = 1  pc = 2  pc = 3  pc = 4  pc = 5  pc = 6  pc = 7  pc = 8  pc = 9  pc = 10  pc = 11  pc = 12  pc = 13  pc = 14  pc = 15  pc = 16  pc = 17  pc = 18  pc = 19  pc = 20  pc = 21  pc = 22  pc = 23  pc = 24  pc = 25  pc = 26  pc = 27
 pc = 28  pc = 29  pc = 30  pc = 31  pc = 32  pc = 33  pc = 34  pc = 35  pc = 36  pc = 37  pc = 38  pc = 39  pc = 40  pc = 41  pc = 42  pc = 43  pc = 44  pc = 45  pc = 46  pc = 47  pc = 48  pc = 49  pc = 50  pc = 51  pc = 52  pc = 53  pc =
 54  pc = 55  pc = 56  pc = 57  pc = 58  pc = 59  pc = 60  pc = 61  pc = 62  pc = 63  pc = 64  pc = 65  pc = 66  pc = 67  pc = 68  pc = 69  pc = 70  pc = 71  pc = 72  pc = 73  pc = 74  pc = 75  pc = 76  pc = 77  pc = 78  pc = 79  pc = 80  p
c = 81  pc = 82  pc = 83  pc = 84  pc = 85  pc = 86  pc = 87  pc = 88  pc = 89  pc = 90  pc = 91  pc = 92  pc = 93  pc = 94  pc = 95  pc = 96  pc = 97  pc = 98  pc = 99  pi = 0  pi = 1  pi = 2  pi = 3  pi = 4  pi = 5  pi = 6  pi = 7  pi = 8
 pi = 9  pi = 10  pi = 11  pi = 12  pi = 13  pi = 14  pi = 15  pi = 16  pi = 17  pi = 18  pi = 19  pi = 20  pi = 21  pi = 22  pi = 23  pi = 24  pi = 25  pi = 26  pi = 27  pi = 28  pi = 29  pi = 30  pi = 31  pi = 32  pi = 33  pi = 34  pi =
35  pi = 36  pi = 37  pi = 38  pi = 39  pi = 40  pi = 41  pi = 42  pi = 43  pi = 44  pi = 45  pi = 46  pi = 47  pi = 48  pi = 49  pi = 50  pi = 51  pi = 52  pi = 53  pi = 54  pi = 55  pi = 56  pi = 57  pi = 58  pi = 59  pi = 60  pi = 61  pi
 = 62  pi = 63  pi = 64  pi = 65  pi = 66  pi = 67  pi = 68  pi = 69  pi = 70  pi = 71  pi = 72  pi = 73  pi = 74  pi = 75  pi = 76  pi = 77  pi = 78  pi = 79  pi = 80  pi = 81  pi = 82  pi = 83  pi = 84  pi = 85  pi = 86  pi = 87  pi = 88
 pi = 89  pi = 90  pi = 91  pi = 92  pi = 93  pi = 94  pi = 95  pi = 96  pi = 97  pi = 98  pi = 99  pf = 0.000000  pf = 1.000000  pf = 2.000000  pf = 3.000000  pf = 4.000000  pf = 5.000000  pf = 6.000000  pf = 7.000000  pf = 8.000000  pf =
9.000000  pf = 10.000000  pf = 11.000000  pf = 12.000000  pf = 13.000000  pf = 14.000000  pf = 15.000000  pf = 16.000000  pf = 17.000000  pf = 18.000000  pf = 19.000000  pf = 20.000000  pf = 21.000000  pf = 22.000000  pf = 23.000000  pf = 2
4.000000  pf = 25.000000  pf = 26.000000  pf = 27.000000  pf = 28.000000  pf = 29.000000  pf = 30.000000  pf = 31.000000  pf = 32.000000  pf = 33.000000  pf = 34.000000  pf = 35.000000  pf = 36.000000  pf = 37.000000  pf = 38.000000  pf = 3
9.000000  pf = 40.000000  pf = 41.000000  pf = 42.000000  pf = 43.000000  pf = 44.000000  pf = 45.000000  pf = 46.000000  pf = 47.000000  pf = 48.000000  pf = 49.000000  pf = 50.000000  pf = 51.000000  pf = 52.000000  pf = 53.000000  pf = 5
4.000000  pf = 55.000000  pf = 56.000000  pf = 57.000000  pf = 58.000000  pf = 59.000000  pf = 60.000000  pf = 61.000000  pf = 62.000000  pf = 63.000000  pf = 64.000000  pf = 65.000000  pf = 66.000000  pf = 67.000000  pf = 68.000000  pf = 6
9.000000  pf = 70.000000  pf = 71.000000  pf = 72.000000  pf = 73.000000  pf = 74.000000  pf = 75.000000  pf = 76.000000  pf = 77.000000  pf = 78.000000  pf = 79.000000  pf = 80.000000  pf = 81.000000  pf = 82.000000  pf = 83.000000  pf = 8
4.000000  pf = 85.000000  pf = 86.000000  pf = 87.000000  pf = 88.000000  pf = 89.000000  pf = 90.000000  pf = 91.000000  pf = 92.000000  pf = 93.000000  pf = 94.000000  pf = 95.000000  pf = 96.000000  pf = 97.000000  pf = 98.000000  pf = 9
9.000000  pd = 0.000000  pd = 1.000000  pd = 2.000000  pd = 3.000000  pd = 4.000000  pd = 5.000000  pd = 6.000000  pd = 7.000000  pd = 8.000000  pd = 9.000000  pd = 10.000000  pd = 11.000000  pd = 12.000000  pd = 13.000000  pd = 14.000000
pd = 15.000000  pd = 16.000000  pd = 17.000000  pd = 18.000000  pd = 19.000000  pd = 20.000000  pd = 21.000000  pd = 22.000000  pd = 23.000000  pd = 24.000000  pd = 25.000000  pd = 26.000000  pd = 27.000000  pd = 28.000000  pd = 29.000000
pd = 30.000000  pd = 31.000000  pd = 32.000000  pd = 33.000000  pd = 34.000000  pd = 35.000000  pd = 36.000000  pd = 37.000000  pd = 38.000000  pd = 39.000000  pd = 40.000000  pd = 41.000000  pd = 42.000000  pd = 43.000000  pd = 44.000000
pd = 45.000000  pd = 46.000000  pd = 47.000000  pd = 48.000000  pd = 49.000000  pd = 50.000000  pd = 51.000000  pd = 52.000000  pd = 53.000000  pd = 54.000000  pd = 55.000000  pd = 56.000000  pd = 57.000000  pd = 58.000000  pd = 59.000000
pd = 60.000000  pd = 61.000000  pd = 62.000000  pd = 63.000000  pd = 64.000000  pd = 65.000000  pd = 66.000000  pd = 67.000000  pd = 68.000000  pd = 69.000000  pd = 70.000000  pd = 71.000000  pd = 72.000000  pd = 73.000000  pd = 74.000000
pd = 75.000000  pd = 76.000000  pd = 77.000000  pd = 78.000000  pd = 79.000000  pd = 80.000000  pd = 81.000000  pd = 82.000000  pd = 83.000000  pd = 84.000000  pd = 85.000000  pd = 86.000000  pd = 87.000000  pd = 88.000000  pd = 89.000000
pd = 90.000000  pd = 91.000000  pd = 92.000000  pd = 93.000000  pd = 94.000000  pd = 95.000000  pd = 96.000000  pd = 97.000000  pd = 98.000000  pd = 99.000000
C:\Users\lyn69\OneDrive\바탕 화면\cording\x64\Debug\cording.exe(프로세스 9232개)이(가) 종료되었습니다(코드: 0개).
디버깅이 중지될 때 콘솔을 자동으로 닫으려면 [도구] -> [옵션] -> [디버깅] > [디버깅이 중지되면 자동으로 콘솔 닫기]를 사용하도록 설정합니다.
이 창을 닫으려면 아무 키나 누르세요...
```

동적 메모리 공간 실습 코드는 동적 메모리 공간이 잘 작동되는지를 확인 해보기 위해서 for문을 사용하여 값을 넣어서 출력하였다. 동적 메모리 공간을 할당해서 변수형마다 결과값이 잘 나오는 것을 확인할 수 있다.

3.(13) 코드 및 주석

```
#include <stdio.h>
```

```
#include <memory.h>
#include <stdlib.h>
void main()
{
	char* pc; //문자열 포인터 pc 선언
	int* pi; //정수형 포인터 pi 선언
	float* pf; //float형 포인터 pf 선언
	double* pd; //double형 포인터 pd 선언
	pc = (char*)malloc(100 * sizeof(char)); //크기가 100인 동적 메모리 할당
	pi = (int*)malloc(100 * sizeof(int)); //크기가 100인 동적 메모리 할당
	pf = (float*)malloc(100 * sizeof(float)); //크기가 100인 동적 메모리 할당
	pd = (double*)malloc(100 * sizeof(double)); //크기가 100인 동적 메모리 할당

	for (int i = 0; i < 100; i++) //결과값 확인을 위해서 0부터 99까지의 수를 넣는다.
	{
		pc[i] = i;
		pi[i] = i;
		pf[i] = i;
		pd[i] = i;
	}
	for (int i = 0; i < 100; i++)
	{
		printf("pc = %d  ", *(pc + i));
	}
	for (int i = 0; i < 100; i++)
	{
		printf("pi = %d  ", *(pi + i));
	}
	for (int i = 0; i < 100; i++)
	{
		printf("pf = %f  ", *(pf + i));
	}
	for (int i = 0; i < 100; i++)
	{
		printf("pd = %f  ", *(pd + i));
	}
}
```

3.(13) 출력화면 및 설명

```
pc = 0  pc = 1  pc = 2  pc = 3  pc = 4  pc = 5  pc = 6  pc = 7  pc = 8  pc = 9  pc = 10  pc = 11  pc = 12  pc = 13  pc = 14  pc = 15  pc = 16  pc = 17  pc = 18  pc = 19  pc = 20  pc = 21  pc = 22  pc = 23  pc = 24  pc = 25  pc = 26  pc = 27
  pc = 28  pc = 29  pc = 30  pc = 31  pc = 32  pc = 33  pc = 34  pc = 35  pc = 36  pc = 37  pc = 38  pc = 39  pc = 40  pc = 41  pc = 42  pc = 43  pc = 44  pc = 45  pc = 46  pc = 47  pc = 48  pc = 49  pc = 50  pc = 51  pc = 52  pc = 53  pc =
 54  pc = 55  pc = 56  pc = 57  pc = 58  pc = 59  pc = 60  pc = 61  pc = 62  pc = 63  pc = 64  pc = 65  pc = 66  pc = 67  pc = 68  pc = 69  pc = 70  pc = 71  pc = 72  pc = 73  pc = 74  pc = 75  pc = 76  pc = 77  pc = 78  pc = 79  pc = 80  p
c = 81  pc = 82  pc = 83  pc = 84  pc = 85  pc = 86  pc = 87  pc = 88  pc = 89  pc = 90  pc = 91  pc = 92  pc = 93  pc = 94  pc = 95  pc = 96  pc = 97  pc = 98  pc = 99  pi = 0  pi = 1  pi = 2  pi = 3  pi = 4  pi = 5  pi = 6  pi = 7  pi = 8
  pi = 9  pi = 10  pi = 11  pi = 12  pi = 13  pi = 14  pi = 15  pi = 16  pi = 17  pi = 18  pi = 19  pi = 20  pi = 21  pi = 22  pi = 23  pi = 24  pi = 25  pi = 26  pi = 27  pi = 28  pi = 29  pi = 30  pi = 31  pi = 32  pi = 33  pi = 34  pi =
35  pi = 36  pi = 37  pi = 38  pi = 39  pi = 40  pi = 41  pi = 42  pi = 43  pi = 44  pi = 45  pi = 46  pi = 47  pi = 48  pi = 49  pi = 50  pi = 51  pi = 52  pi = 53  pi = 54  pi = 55  pi = 56  pi = 57  pi = 58  pi = 59  pi = 60  pi = 61  pi
 = 62  pi = 63  pi = 64  pi = 65  pi = 66  pi = 67  pi = 68  pi = 69  pi = 70  pi = 71  pi = 72  pi = 73  pi = 74  pi = 75  pi = 76  pi = 77  pi = 78  pi = 79  pi = 80  pi = 81  pi = 82  pi = 83  pi = 84  pi = 85  pi = 86  pi = 87  pi = 88
 pi = 89  pi = 90  pi = 91  pi = 92  pi = 93  pi = 94  pi = 95  pi = 96  pi = 97  pi = 98  pi = 99  pf = 0.000000  pf = 1.000000  pf = 2.000000  pf = 3.000000  pf = 4.000000  pf = 5.000000  pf = 6.000000  pf = 7.000000  pf = 8.000000  pf =
9.000000  pf = 10.000000  pf = 11.000000  pf = 12.000000  pf = 13.000000  pf = 14.000000  pf = 15.000000  pf = 16.000000  pf = 17.000000  pf = 18.000000  pf = 19.000000  pf = 20.000000  pf = 21.000000  pf = 22.000000  pf = 23.000000  pf = 2
4.000000  pf = 25.000000  pf = 26.000000  pf = 27.000000  pf = 28.000000  pf = 29.000000  pf = 30.000000  pf = 31.000000  pf = 32.000000  pf = 33.000000  pf = 34.000000  pf = 35.000000  pf = 36.000000  pf = 37.000000  pf = 38.000000  pf = 3
9.000000  pf = 40.000000  pf = 41.000000  pf = 42.000000  pf = 43.000000  pf = 44.000000  pf = 45.000000  pf = 46.000000  pf = 47.000000  pf = 48.000000  pf = 49.000000  pf = 50.000000  pf = 51.000000  pf = 52.000000  pf = 53.000000  pf = 5
4.000000  pf = 55.000000  pf = 56.000000  pf = 57.000000  pf = 58.000000  pf = 59.000000  pf = 60.000000  pf = 61.000000  pf = 62.000000  pf = 63.000000  pf = 64.000000  pf = 65.000000  pf = 66.000000  pf = 67.000000  pf = 68.000000  pf = 6
9.000000  pf = 70.000000  pf = 71.000000  pf = 72.000000  pf = 73.000000  pf = 74.000000  pf = 75.000000  pf = 76.000000  pf = 77.000000  pf = 78.000000  pf = 79.000000  pf = 80.000000  pf = 81.000000  pf = 82.000000  pf = 83.000000  pf = 8
4.000000  pf = 85.000000  pf = 86.000000  pf = 87.000000  pf = 88.000000  pf = 89.000000  pf = 90.000000  pf = 91.000000  pf = 92.000000  pf = 93.000000  pf = 94.000000  pf = 95.000000  pf = 96.000000  pf = 97.000000  pf = 98.000000  pf = 9
9.000000  pd = 0.000000  pd = 1.000000  pd = 2.000000  pd = 3.000000  pd = 4.000000  pd = 5.000000  pd = 6.000000  pd = 7.000000  pd = 8.000000  pd = 9.000000  pd = 10.000000  pd = 11.000000  pd = 12.000000  pd = 13.000000  pd = 14.000000
pd = 15.000000  pd = 16.000000  pd = 17.000000  pd = 18.000000  pd = 19.000000  pd = 20.000000  pd = 21.000000  pd = 22.000000  pd = 23.000000  pd = 24.000000  pd = 25.000000  pd = 26.000000  pd = 27.000000  pd = 28.000000  pd = 29.000000
pd = 30.000000  pd = 31.000000  pd = 32.000000  pd = 33.000000  pd = 34.000000  pd = 35.000000  pd = 36.000000  pd = 37.000000  pd = 38.000000  pd = 39.000000  pd = 40.000000  pd = 41.000000  pd = 42.000000  pd = 43.000000  pd = 44.000000
pd = 45.000000  pd = 46.000000  pd = 47.000000  pd = 48.000000  pd = 49.000000  pd = 50.000000  pd = 51.000000  pd = 52.000000  pd = 53.000000  pd = 54.000000  pd = 55.000000  pd = 56.000000  pd = 57.000000  pd = 58.000000  pd = 59.000000
pd = 60.000000  pd = 61.000000  pd = 62.000000  pd = 63.000000  pd = 64.000000  pd = 65.000000  pd = 66.000000  pd = 67.000000  pd = 68.000000  pd = 69.000000  pd = 70.000000  pd = 71.000000  pd = 72.000000  pd = 73.000000  pd = 74.000000
pd = 75.000000  pd = 76.000000  pd = 77.000000  pd = 78.000000  pd = 79.000000  pd = 80.000000  pd = 81.000000  pd = 82.000000  pd = 83.000000  pd = 84.000000  pd = 85.000000  pd = 86.000000  pd = 87.000000  pd = 88.000000  pd = 89.000000
pd = 90.000000  pd = 91.000000  pd = 92.000000  pd = 93.000000  pd = 94.000000  pd = 95.000000  pd = 96.000000  pd = 97.000000  pd = 98.000000  pd = 99.000000
C:\Users\lyn69\OneDrive\바탕 화면\cording\x64\Debug\cording.exe(프로세스 9232개)이(가) 종료되었습니다(코드: 0개).
디버깅이 중지될 때 콘솔을 자동으로 닫으려면 [도구] -> [옵션] -> [디버깅] > [디버깅이 중지되면 자동으로 콘솔 닫기]를 사용하도록 설정합니다.
이 창을 닫으려면 아무 키나 누르세요...
```

3(12) 코드에서 free()함수를 제거하였다. 결과는 달라지지 않는 것을 확인할 수 있다. free함수는 동적 메모리를 해제하기 위해서 사용하는 것이라서 없다고 결과에는 영향을 주지 않는다. 하지만 free함수를 없애면 동적메모리가 그대로 남아있기 때문에 메모리에 문제가 생길 수도 있다.

3.(14~15) 코드 및 주석

```
#include <stdio.h>
#include <memory.h>
#include <stdlib.h>
void main()
{
        char* pc; //문자열 포인터 pc 선언
        int* pi; //정수형 포인터 pi 선언
        float* pf; //float형 포인터 pf 선언
        double* pd; //double형 포인터 pd 선언
        pc = (char*)malloc(100 * sizeof(char)); //크기가 100인 동적 메모리 할당
        pi = (int*)malloc(100 * sizeof(int)); //크기가 100인 동적 메모리 할당
        pf = (float*)malloc(100 * sizeof(float)); //크기가 100인 동적 메모리 할당
        pd = (double*)malloc(100 * sizeof(double)); //크기가 100인 동적 메모리 할당

        for (int i = 0; i < 100; i++) //결과값 확인을 위해서 0부터 99까지의 수를 넣는다.
        {
                pc[i] = i;
                pi[i] = i;
                pf[i] = i;
                pd[i] = i;
        }
        printf("pc = %lu\n", pc); //pc의 메모리주소를 출력한다.
        printf("pc+1 = %lu\n", pc + 1); //pc+1의 메모리주소를 출력한다.
        printf("pc-1 = %lu\n", pc - 1); //pc-1의 메모리주소를 출력한다.

        printf("pi = %lu\n", pi); //pi의 메모리주소를 출력한다.
        printf("pi+1 = %lu\n", pi + 1); //pi+1의 메모리주소를 출력한다.
        printf("pi-1 = %lu\n", pi - 1); //pi -1 의 메모리주소를 출력한다.

        printf("pf = %lu\n", pf); //pf의 메모리주소를 출력한다.
        printf("pf+1 = %lu\n", pf + 1); //pf+1의 메모리주소를 출력한다.
        printf("pf-1 = %lu\n", pf - 1); //pf-1의 메모리주소를 출력한다.
        printf("pd = %lu\n", pd); //pd의 메모리주소를 출력한다.
```

```
        printf("pd+1 = %lu\n", pd + 1); //pd+1의 메모리주소를 출력한다.
        printf("pd-1 = %lu\n", pd - 1); //pd-1의 메모리주소를 출력한다.

        free(pc);
        free(pi);
        free(pf);
        free(pd);
}
```

3.(14~15) 출력화면 및 설명

```
pc = 1410374528
pc+1 = 1410374529
pc-1 = 1410374527
pi = 1410395264
pi+1 = 1410395268
pi-1 = 1410395260
pf = 1410395728
pf+1 = 1410395732
pf-1 = 1410395724
pd = 1410404416
pd+1 = 1410404424
pd-1 = 1410404408

C:\Users\lym69\OneDrive\바탕 화면\cording\x64\Debug\cording.exe(프로세스 4340개)이(가) 종료되었습니다(코드: 0개).
디버깅이 중지될 때 콘솔을 자동으로 닫으려면 [도구] -> [옵션] -> [디버깅] > [디버깅이 중지되면 자동으로 콘솔 닫기]를 사용하도록 설정합니다.
이 창을 닫으려면 아무 키나 누르세요...
```

메모리 주소를 출력하는 과정에서 결과값에 차이가 나타나는 모습을 볼수 있다. pc의 경우 문자형 변수이기 때문에 크기가 1이라서 +1과-1을 했을 때 메모리주소 값의 차이가 1이지만, pi와 pf는 int형과 float형이라서 4씩 차이가 나고, double형은 8씩 차이가 나는 걸 알 수 있다. 따라서 이 결과값을 보고 변수형의 크기에 따라서 메모리 주소 값의 차이를 확인 할 수 있다.

3.(16) 코드 및 주석

```
#include <stdio.h>
#include <memory.h>
#include <stdlib.h>

typedef struct test  //test라는 구조체 선언
{
        int data;
        struct test* link; //자체참조를 이용한 연결리스트의 형태인 것을 알수 있다.
} test_str;
void main()
{
        int i;
        test_str* ptest;
        ptest = (double*)malloc(sizeof(test_str)); //동적메모리 할당
        ptest->data = 1; //구조체의 data에 1 값을 넣는다.
        printf("%d\n", ptest->data); // 구조체의 data값을 출력한다.

        free(ptest);
}
```

3.(16) 출력화면 및 설명

```
1

C:\Users\lym69\OneDrive\바탕 화면\cording\x64\Debug\cording.exe(프로세스 19648개)이(가) 종료되었습니다(코드: 0개).
디버깅이 중지될 때 콘솔을 자동으로 닫으려면 [도구] -> [옵션] -> [디버깅] > [디버깅이 중지되면 자동으로 콘솔 닫기]를 사용하도록 설정합니다.
이 창을 닫으려면 아무 키나 누르세요...
```

test라는 구조체를 만들어서 ptest에 동적메모리 할당을 한 이후 test구조체의 data에 1값을 넣고 출력을 한 결과이다. 구조체의 형태는 자체참조하는 연결리스트의 형태이지만, 구조체 안에 데이터를 저장해보라는 의도에 맞게 구조체 안에 데이터를 넣어서 확인하였다. 코딩을 수정을 거치면 연결리스트로 사용이 가능할 것으로 보인다.

3.(17) 코드 및 주석

```
#include <stdio.h>
#include <memory.h>
#include <stdlib.h>

void main()
{
	int* pi; //정수형 포인터 pi 선언
	pi = (int*)malloc(100 * sizeof(char)); //문자형 크기 100을 가지는 메모리 할당
	for (int i = 0; i < 100; i++) //for문을 사용해서 내용물 확인
	{
		pi[i] = i;
		printf("i= %d. ",pi[i]);
	}
	free(pi);
}
```

3.(17) 출력화면 및 설명

```
i= 0. i= 1. i= 2. i= 3. i= 4. i= 5. i= 6. i= 7. i= 8. i= 9. i= 10. i= 11. i= 12. i= 13. i= 14. i= 15. i= 16. i= 17. i= 1
8. i= 19. i= 20. i= 21. i= 22. i= 23. i= 24. i= 25. i= 26. i= 27. i= 28. i= 29. i= 30. i= 31. i= 32. i= 33. i= 34. i= 35
. i= 36. i= 37. i= 38. i= 39. i= 40. i= 41. i= 42. i= 43. i= 44. i= 45. i= 46. i= 47. i= 48. i= 49. i= 50. i= 51. i= 52.
 i= 53. i= 54. i= 55. i= 56. i= 57. i= 58. i= 59. i= 60. i= 61. i= 62. i= 63. i= 64. i= 65. i= 66. i= 67. i= 68. i= 69.
i= 70. i= 71. i= 72. i= 73. i= 74. i= 75. i= 76. i= 77. i= 78. i= 79. i= 80. i= 81. i= 82. i= 83. i= 84. i= 85. i= 86. i
= 87. i= 88. i= 89. i= 90. i= 91. i= 92. i= 93. i= 94. i= 95. i= 96. i= 97. i= 98. i= 99.
C:\Users\lym69\OneDrive\바탕 화면\cording\x64\Debug\cording.exe(프로세스 25716개)이(가) 종료되었습니다(코드: -1개).
디버깅이 중지될 때 콘솔을 자동으로 닫으려면 [도구] -> [옵션] -> [디버깅] > [디버깅이 중지되면 자동으로 콘솔 닫기]를 사용하도록 설정합니다.
이 창을 닫으려면 아무 키나 누르세요...
```

동적 메모리를 할당할 때, 변수형과 변수의 크기를 정할 때 같은 것으로 해줘야 문제가 생기지 않지만, 다르게 만든 후 결과를 확인하고 있다. 결과는 문제가 생기지 않고 결과값이 잘 출력이 되었다. 하지만 결과값이 제대로 잘 출력이 되었다고 이 코드가 문제가 없는 것은 아니다. int가 가져야 할 크기보다 작은 크기인 char 크기로 만들었기 때문에 메모리 오버플로우 현상이 일어날 수 있다.

```
int* pi;
pi = (int*)malloc(100 * sizeof(char));
for (int i = 1000000; i < 1000010; i++)
{
    pi[i] = i;
    printf("i= %d. ",pi[i]);
}
free(pi);
```

```
    설명
4   for 루프가 잘못 정의되었습니다.  Loop 본문이 실행되지 않습니다.
0   인덱스 '100099'이(가) 다음 비스택 버퍼에 대해 유효한 인덱스 범위 '0' - '24'을(를) 벗어납니다. 'pi'.
0   인덱스 '100099'이(가) 다음 비스택 버퍼에 대해 유효한 인덱스 범위 '0' - '24'을(를) 벗어납니다. 'pi'.
```

너무 큰 값을 넣었을 때 오버플로우 현상이 일어나는 것을 확인할 수 있다. 따라서 정수형과 크기를 같은 형으로 맞춰줄 필요가 있다는 것을 확인할 수 있다.

135p .17 코드 및 주석

```
#include <stdio.h>
#include <memory.h>
#include <stdlib.h>
#include <time.h>

#define _CRT_SECURE_NO_WARNINGS
#pragma warning(disable:4996)

typedef struct Node  //이중 연결리스트 정의
{
    struct Node* llink;
    int data;
    struct Node* rlink;
}Node;

Node* head = NULL; // 시작노드

void addNode(int data)
{
    Node* newNode = (Node*)malloc(sizeof(Node)); //동적할당
    newNode->data = data;
    newNode->llink = NULL;
    newNode->rlink = NULL;

    if (head == NULL) //연결리스트가 비어있는 경우
        {
        head = newNode; //새 노드를 시작노드로 지정, 오른쪽 링크,왼쪽 링크 모두 본인으로 지정
        newNode->llink = newNode;
        newNode->rlink = newNode;
    }
    else //노드를 추가하기
        {
        newNode->rlink = head; // 새 노드의 오른쪽 링크를 기존 시작 노드로 지정
        newNode->llink = head->llink; // 새 노드의 왼쪽 링크를 기존 끝 노드로 지정
        head->llink->rlink = newNode; // 기존 끝 노드의 오른쪽 링크를 새 노드로 지정
        head->llink = newNode; // 기존 시작 노드의 왼쪽 링크를 새 노드로 지정
```

```
    }
}

void printList(char name[][30], int number) // 연결리스트를 출력함
{
    Node* current = head;
    int i=0;
    while(i<number) //남은 학생 수만큼 출력하기
    {
        printf("남은 학생: %s\n", name[current->data]);
        current = current->rlink;
                i++;
    }
}
void deleteNode1(int n, char name[][30]) //시계방향으로 찾아가서 제거
{
    Node* current = head;
    for (int i = 0; i < n; i++) //n값 만큼 반복

    {
        current = current->rlink; //시계방향으로 이동
    }
    printf("제거된 학생의 이름:%s\n", name[current->data]); //연결리스트에 있는 번호에 맞는 배열 학생 이름 출력
    //노드 제거하기
    current->llink->rlink = current->rlink;
    current->rlink->llink = current->llink;
    head =current->rlink; //제거된 노드 위치를 기억하게 한다.
}
void deleteNode2(int n, char name[][30]) //역시계방향으로 찾아가서 제거
{
    Node* current = head;
    for (int i = 0; i <n; i++) //n값 만큼 반복

    {
        current = current->llink; //역시계방향으로 이동
    }
    printf("제거된 학생의 이름:%s\n", name[current->data]); //연결리스트에 있는 번호에 맞는 배열 학생 이름 출력
    //노드 제거하기
    current->llink->rlink = current->rlink;
    current->rlink->llink = current->llink;
    head =current->rlink; //제거된 노드 위치를 기억하게 한다.
}
```

```
int main()
{
    int number = 0, n = 0, value = 0;//학생 수 number, 숫자 n, 시계방향 역시계방향  구분을 위한 value
    char name[100][30] = { 0 }; //학생 수의 최대는 100명에 이름의 최대는 30으로 고정해서 저장하는 배열
    srand(time(0));  //난수값이 시간에 따라 변화하도록 하였다.

    printf("학생 수와 숫자 하나를 입력하시오: ");
    scanf("%d %d", &number, &n);
    int random = rand() % number; //난수값을 random에 넣는 과정
    for (int i = 0; i < number; i++) //학생의 수만큼 반복
    {
        printf("학생의 이름을 입력하세요:");
        scanf("%s", name[i]); //학생 이름을 입력받아서 배열에 저장
        addNode(i); //학생의 번호를 저장함
    }
        deleteNode1(random+n, name); //임의의 숫자로부터 시계방향 n번째 사람 제거
    number--; //학생 수가 한명 줄어든다.
    printList(name,number); //남은 학생 수 출력
    value = 1; //역시계방향으로 변경
    while (1) //학생수가 1명만 남을때까지 반복
    {
        if (value == 1) //역시계방향
        {
            deleteNode2(n, name); //임의의 숫자로부터 시계방향 n번째 사람 제거
                        number--; //학생 수가 한명 줄어든다.
                printList(name,number); //남은 학생수 출력
            value = 0; //시계방향으로 변경
            if (number == 1)//한명 남으면 끝내기
            {
                break;
            }

        }
        if (value == 0) //시계방향
        {
            deleteNode1(n, name); //임의의 숫자로부터 시계방향 n번째 사람 제거
                        number--; //학생 수 한명 줄어든다.
                printList(name,number); //남은 학생 수 출력
            value = 1; // 역시계방향으로 바꿈
            if (number == 1) //한명 남으면 끝내기
            {
                break;
```

```
            }

        }
    }
    printf("\n\n최종");
    printList(name,number); //남은 학생 수가 한명이기 때문에 최종 학생이 된다.

    return 0;
}
```

135p.17 출력화면 및 설명

```
학생 수와 숫자 하나를 입력하시오: 5 3
학생의 이름을 입력하세요:한결
학생의 이름을 입력하세요:준형
학생의 이름을 입력하세요:태균
학생의 이름을 입력하세요:승우
학생의 이름을 입력하세요:상준
제거된 학생의 이름:한결
남은 학생: 준형
남은 학생: 태균
남은 학생: 승우
남은 학생: 상준
제거된 학생의 이름:태균
남은 학생: 승우
남은 학생: 상준
남은 학생: 준형
제거된 학생의 이름:승우
남은 학생: 상준
남은 학생: 준형
제거된 학생의 이름:준형
남은 학생: 상준


최종남은 학생: 상준
```

학생 수와 숫자 n을 입력받는다. 학생의 번호를 연결리스트에 받아서 저장하고, 배열을 만들어서 학생 번호와 매칭이 되게 이름을 저장했다. 그 후 임의의 숫자와 n값으로 시계방향으로 이동하여 노드를 제거하였다. 그 이후로는 while문을 이용하여 학생 수가 1명이 남을 때까지 시계방향, 역시계방향을 반복하여 학생을 제거하는 식을 만들었다.

이 문제를 해결하기 위해서 시계방향으로 이동해서 노드를 제거하는 함수와 역 시계방향으로 이동해서 노드를 제거하는 함수 두 가지로 나눠서 수행했다. head는 노드 생성 이후 노드를 제거할 때, 제거한

노드 위치를 기억하게끔 사용하였다. 학생을 출력하는 함수는 연결리스트에 저장된 학생 번호 데이터 값과 일치하는 배열을 출력하여 학생의 이름을 출력하게 하였다.